Memory Diagrams

Name: _____ Date: _____

How we draw memory. Objects (arrays and records) live on the **heap**. Each one gets an address, `@1001`, `@1002`, ..., in the order it is created. Names live in **frames** on the **stack**: the top level is at the bottom, and each method call gets a frame on top of the frame that called it, labeled with the method call and the address (`@addr`) of the object the method was called on. A frame holds the method's parameters and variables. Fields stay in their object on the heap. When a value changes, cross out the old one: ~~350~~ 475. Leave a line empty if nothing goes there.

Filling in memory

Q1. Fill in memory at the moment `total` becomes `475`.

```
record Receipt(int[] pricesCents) {
  int totalCents() {
    int total = 0;
    for(int c: pricesCents) {
      total = total + c;
    }
    return total;
  }
}
int[] prices = { 350, 125, 499 };
Receipt lunch = new Receipt(prices);
Receipt dinner = new Receipt(prices);
int lunchTotal = lunch.totalCents();
```

NAMES (THE STACK)

`lunch.totalCents()` on _____
`total` = _____
`c` = _____

top level
`prices` = _____
`lunch` = _____
`dinner` = _____

HEAP

`@1001`: _____
`@1002`: _____
`@1003`: _____
`@1004`: _____

Q2. Why is there no `lunchTotal` in the top-level frame yet? And why is there no `pricesCents` in the `lunch.totalCents()` frame?

Which program?

```
record Menu(int[] pricesCents) {
  int[] discounted() {
    int[] result = new int[pricesCents.length];
    int index = 0;
    for(int p: pricesCents) {
      result[index] = p * 90 / 100;
      index += 1;
    }
    return result;
  }
  void discountInPlace() {
    int index = 0;
    for(int p: pricesCents) {
      pricesCents[index] = p * 90 / 100;
      index += 1;
    }
  }
}
```

This is memory after a program that uses `Menu` finishes (frames of finished calls are left out):

NAMES (THE STACK)	HEAP

top level
`full` = `@1001`
`menu` = `@1002`
`sale` = `@1003`
	`@1001`: `[900, 500, 300]`
	`@1002`: `Menu[pricesCents=@1001]`
	`@1003`: `[810, 450, 270]`

A

```
int[] full = { 900, 500, 300 };
Menu menu = new Menu(full);
int[] sale = menu.discounted();
```

B

```
int[] full = { 900, 500, 300 };
Menu menu = new Menu(full);
int[] sale = full;
void main() {
  menu.discountInPlace();
}
```

C

```
int[] full = { 900, 500, 300 };
Menu menu = new Menu(full);
int[] sale = { 810, 450, 270 };
```

Q3. Which programs end with exactly this memory? Write the letter of every one that does. _____

Q4. For each program you did not list, what would its diagram show instead?

Q5. If you listed more than one program in Q3, pick two of them. Draw or describe one thing that could appear in a memory diagram while one is running that could never appear while the other one is running.

Drawing memory

All three programs on this side use this record:

```
record Gradebook(int[] scores) {
  int[] curved() {
    int[] result = new int[scores.length];
    int index = 0;
    for(int s: scores) {
      result[index] = s + 5;
      index += 1;
    }
    return result;
  }
  void curveInPlace() {
    int index = 0;
    for(int s: scores) {
      scores[index] = s + 5;
      index += 1;
    }
  }
}
```

Q6. Fill in memory at the moment the loop body is about to run for the **3rd** time.

```
int[] quiz = { 80, 92, 75 };
Gradebook book = new Gradebook(quiz);
void main() {
  book.curveInPlace();
}
```

NAMES (THE STACK)

`book.curveInPlace()` on _____
`index` = _____
`s` = _____

`main()`

top level
`quiz` = _____
`book` = _____

HEAP

`@1001`: [_____, _____, _____]
`@1002`: `Gradebook[scores=`_____`]`

Q7. Draw memory just before `return result;` runs. Each line is one name or one heap slot.

```
int[] quiz = { 80, 92, 75 };
Gradebook book = new Gradebook(quiz);
int[] curvedQuiz = book.curved();
```

NAMES (THE STACK)

_____
_____
_____
_____
_____

top level
_____
_____
_____

HEAP

_____
_____
_____
_____

Q8. Draw memory when the program reaches ★.

```
int[] quiz = { 80, 92, 75 };
Gradebook morning = new Gradebook(quiz);
Gradebook evening = new Gradebook(quiz);
void main() {
  evening.curveInPlace();
  int[] copy = morning.curved();
  // ★
}
```



Q9. At ★, what is `morning.scores[0]`? _____ What is `copy[0]`? _____

Q10. `evening.curveInPlace()` was supposed to change only the evening section's scores. Change the program so that `morning`'s scores stay `80, 92, 75`.

Q11. Redraw only the heap at ★ for your fixed program.

